

Borrador

Modulo de administrador:

GET: /api/administrador/reporteactivos

Response: {Número de alumnos activos: [valor], numero de alumnos inactivos: [valor]}

Lógica de los campos: Conteo de alumnos activos y alumnos inactivos

Query de referencia:

```
SELECT
    u.id,
    u.name,
    u.email,
    s.student_code,
    s.enrollment_status
FROM
    USERS u
JOIN
    STUDENTS s ON u.id = s.user_id
WHERE
    u.status = 'active'
    AND s.enrollment_status = 'active';
```

GET:/api/administrador/reportepagos

Response: {Sumatoria de pagos realizados: [valor], Sumatoria de pagos pendientes: [valor]}

Modulo de autenticación:

POST: /api/auth/login

```
Response: {
  "success": true,
  "message": "Inicio de sesión exitoso",
  "data": {
    "session": {
      "user_id": id,
      "username": name,
      "role": role
    },
    "access_token": token,
    "token_type": "Bearer",
    "expires_in": tiempo_expiracion
  }
}
```

POST: /api/auth/logout

```
Response: {
  "success": true,
  "message": "Sesión cerrada correctamente"
}
```

GET: /api/auth/me

Response: {

```

"success": true,
"data": {
  "user": {
    "id": id,
    "username": name,
    "email": email,
    "name": name,
    "status": "active",
    "last_login_at": fecha_login,
    "created_at": fecha_creacion
  },
  "role": {
    "id": id,
    "name": "student",
    "description": descripcion
  },
}
}

```

Módulo de perfiles:

GET: api/alumnos/me

Response:{Nombre:[Valor], Curso[Valor], Status[Activo/Inactivo]}

GET: /api/alumnos/me/pagos - historial de pagos

Response:{IdAlumno:[Valor], Pagos:[{Fecha:[Date], Descripción:[Valor],
Monto:[Valor]},{Fecha:[Date], Descripción:[Valor], Monto:[Valor]},{Fecha:[Date],
Descripción:[Valor], Monto:[Valor]},{Fecha:[Date], Descripción:[Valor],
Monto:[Valor]},{Fecha:[Date], Descripción:[Valor], Monto:[Valor]}}

Procesamiento de pagos:

POST: /api/pagos/procesar

Response: {StatusPago:[Exitoso/Norealizado] }

GEMINI RESPONSE

<https://gemini.google.com/share/0e14040c658e>

Documentación Técnica de API (FastAPI)

Modelos Pydantic (Esquemas)

Python

```
from pydantic import BaseModel
from typing import List, Optional
from datetime import datetime
```

```
class ReporteActivosResponse(BaseModel):
    numero_alumnos_activos: int
    numero_alumnos_inactivos: int
```

```
class ReportePagosResponse(BaseModel):
    sumatoria_pagos_realizados: float
    sumatoria_pagos_pendientes: float
```

```
class SessionData(BaseModel):
    user_id: str
    username: str
    role: str
```

```
class LoginData(BaseModel):
    session: SessionData
    access_token: str
    token_type: str = "Bearer"
    expires_in: int
```

```
class LoginResponse(BaseModel):
    success: bool
    message: str
    data: LoginData
```

```
class LogoutResponse(BaseModel):
    success: bool
    message: str
```

```
class RoleInfo(BaseModel):
    id: str
    name: str
    description: str
```

```
class UserInfo(BaseModel):
    id: str
    username: str
    email: str
```

```
name: str
status: str
last_login_at: Optional[datetime]
created_at: datetime
```

```
class MeData(BaseModel):
    user: UserInfo
    role: RoleInfo
```

```
class MeResponse(BaseModel):
    success: bool
    data: MeData
```

```
class PerfilAlumnoResponse(BaseModel):
    Nombre: str
    Curso: str
    Status: str
```

```
class PagoHistorial(BaseModel):
    Fecha: datetime
    Descripcion: str
    Monto: float
```

```
class HistorialPagosResponse(BaseModel):
    IdAlumno: str
    Pagos: List[PagoHistorial]
```

```
class ProcesarPagoRequest(BaseModel):
    payment_id: str
    card_number: str
    exp_date: str
    cvc: str
    country: str
```

```
class ProcesarPagoResponse(BaseModel):
    StatusPago: str
```

Rutas (Endpoints)

Módulo: Administrador

Python

```
from fastapi import APIRouter, Depends
```

```
router_admin = APIRouter(prefix="/api/administrador", tags=["Administrador"])
```

```
@router_admin.get("/reporteactivos", response_model=ReporteActivosResponse)
async def obtener_reporte_activos():
```

pass

Mapeo DB: Cuenta **USERS.id** donde **USERS.status** y **STUDENTS.enrollment_status** determinan la agrupación.

Python

```
@router_admin.get("/reportepagos", response_model=ReportePagosResponse)
async def obtener_reporte_pagos():
    pass
```

Mapeo DB: Suma de **PAYMENT_STUDENTS.paid_amount** para realizados y **PAYMENT_STUDENTS.assigned_amount** menos pagados para pendientes, evaluando **PAYMENT_STUDENTS.status**.

Módulo: Autenticación

Python

```
router_auth = APIRouter(prefix="/api/auth", tags=["Autenticación"])
```

```
@router_auth.post("/login", response_model=LoginResponse)
async def login():
    pass
```

Mapeo DB: Validación contra **USERS.username** o **USERS.email** y **USERS.password_hash**. Genera registro en **LOGIN_LOGS**.

Python

```
@router_auth.post("/logout", response_model=LogoutResponse)
async def logout():
    pass
```

```
@router_auth.get("/me", response_model=MeResponse)
async def get_me():
    pass
```

Mapeo DB: Cruce de **USERS** y **ROLES** usando **USERS.role_id**.

Módulo: Perfiles (Alumnos)

Python

```
router_alumnos = APIRouter(prefix="/api/alumnos", tags=["Alumnos"])
```

```
@router_alumnos.get("/me", response_model=PerfilAlumnoResponse)
async def obtener_perfil_alumno():
    pass
```

Mapeo DB: Extrae **USERS.name** , **STUDENTS.enrollment_status** y resolución de curso vía **ENROLLMENTS.course_subject_id**.

Python

```
@router_alumnos.get("/me/pagos", response_model=HistorialPagosResponse)
async def obtener_historial_pagos():
    pass
```

Mapeo DB: Cruce de `STUDENTS.id` con `PAYMENT_STUDENTS` y `PAYMENTS`. Retorna `PAYMENT_STUDENTS.paid_at` , `PAYMENTS.concept` y `PAYMENT_STUDENTS.assigned_amount`.

Módulo: Procesamiento de Pagos

Python

```
router_pagos = APIRouter(prefix="/api/pagos", tags=["Pagos"])
```

```
@router_pagos.post("/procesar", response_model=ProcesarPagoResponse)
async def procesar_pago(request: ProcesarPagoRequest):
    pass
```

Mapeo DB: Actualiza `PAYMENT_STUDENTS.paid_amount` , `PAYMENT_STUDENTS.paid_at` , `PAYMENT_STUDENTS.payment_method` y `PAYMENT_STUDENTS.status`.